APLICACIONES · MODELADO · ALGORITMOS

Diez aplicaciones reales de los problemas NP-hard

Cómo transformar decisiones del mundo real en modelos combinatorios

Modelos, variables, restricciones, objetivos y ejemplos numéricos para estudiantes.

Una continuación de la guía de las 18 reducciones.

Reparto de medicamentos • turnos hospitalarios • cómputo • pruebas de software • servicios de emergencia • configuración • horarios • producción • difusión • conservación

IDEA CENTRAL

Una aplicación suele combinar varias estructuras: seleccionar, cubrir, asignar, ordenar y conectar. Comprender sus interacciones es tan importante como reconocer cada problema por separado.

Una guía de modelado: de la situación real al algoritmo

Este documento continúa la guía de las 18 reducciones. Su objetivo es mostrar, en diez situaciones concretas, cómo identificar decisiones, expresar restricciones y combinar estructuras fundacionales. Son escenarios ilustrativos inspirados en clases de problemas reales; los datos pequeños son inventados para poder revisar las cuentas, y no describen implementaciones de organizaciones particulares.

Cada aplicación incluye un modelo básico, un ejemplo y las condiciones que exigen ampliarlo. Una formulación matemática es una representación de la realidad: su solución puede ser óptima para el modelo y, aun así, resultar inadecuada si faltan restricciones relevantes.

Antes de empezar: cuatro reglas

1. Separar decisiones de datos. La demanda estimada es un dato; elegir dónde instalar un centro es una decisión. Si algo puede cambiarse, no debe tratarse inadvertidamente como una constante.

2. Distinguir restricciones y preferencias. Una incompatibilidad clínica es una condición obligatoria. Una preferencia por cierto turno puede penalizarse. No conviene permitir que una mejora económica compense una infracción obligatoria.

3. Precisar la versión del problema. Mochila 0/1 no permite fraccionar objetos. Coloreo representa conflictos entre pares. Set Cover exige cubrir todo; Maximum Coverage aprovecha un presupuesto para cubrir lo máximo posible. Las variantes modifican tanto el modelo como sus algoritmos.

4. Justificar la dificultad. Usar variables binarias o un solucionador MIP no demuestra que una familia de problemas sea NP-hard. Una justificación válida identifica un caso particular que contiene un problema conocido y respeta sus supuestos. Los ejemplos pequeños pueden ser fáciles aunque la familia general sea difícil.

Notación

Una variable binaria vale 1 si se toma una decisión y 0 en caso contrario. Σ denota suma; $\in$ significa pertenencia; $\leq$ y $\geq$ representan límites. Los índices y sus conjuntos se definen en cada caso. Las variables auxiliares de cobertura requieren especial cuidado: una persona cubierta por dos acciones se cuenta una sola vez.

Las diez aplicaciones

Caso	Aplicación	Estructuras principales
1	Reparto de medicamentos	TSP, rutas con capacidad, selección tipo mochila
2	Turnos de personal hospitalario	Cobertura de conjuntos, conflictos, asignación
3	Trabajos en una plataforma de cómputo	Makespan, empaquetamiento, mochila
4	Selección de pruebas de software	Cobertura máxima, mochila, scheduling
5	Ubicación de servicios de emergencia	Cobertura máxima, localización, asignación
6	Configuración de productos y software	SAT, mochila, MIP
7	Horarios y oferta académica	Coloreo, cobertura, asignación
8	Producción con cambios de herramienta	TSP, scheduling, asignación
9	Campañas de información en redes	Influencia, cobertura, mochila
10	Protección de ecosistemas	Cobertura máxima, mochila, conectividad

1. Reparto de medicamentos con capacidad y horarios

Situación y decisiones

Una distribuidora debe abastecer clínicas desde un depósito. Cada entrega tiene volumen, una ventana de recepción y un tiempo de descarga. Algunos productos requieren cadena de frío. Se decide qué vehículo atiende cada clínica, en qué orden y a qué hora. Las entregas urgentes son obligatorias; otras podrían posponerse si el modelo operativo lo permite.

Datos y estructura fundacional

Sean C las clínicas y K los vehículos. Se conocen demandas q_i, capacidades Q_k, tiempos de viaje τ_{ij}, costos c_{ij}, tiempos de servicio s_i y ventanas $[a_i,b_i]$. El depósito es el nodo 0. Los tiempos deben obtenerse de la red vial, no necesariamente de distancias en línea recta.

Con un vehículo, todas las visitas obligatorias y sin horarios ni capacidad, aparece TSP. Con varios vehículos y capacidad aparece el problema de rutas de vehículos, VRP. Si hay entregas opcionales con beneficio y límite de recursos, surge una selección parecida a mochila, pero su costo depende de la ruta conjunta: no basta sumar costos de viaje individuales.

Variables, restricciones y objetivo

• $x_{ijk} = 1$ si el vehículo k viaja de i a j.

• $y_{ik} = 1$ si k atiende la clínica i; t_{ik} es su hora de llegada.

• Para toda clínica obligatoria: $\Sigma_k\, y_{ik} = 1$.

• Para cada vehículo: $\Sigma_i\, q_i y_{ik} \le Q_k$, suponiendo entregas desde el depósito y sin recargas.

• Las entradas y salidas de cada visita deben coincidir con y_{ik}. Se necesitan restricciones que impidan ciclos desconectados del depósito.

• Si $x_{ijk} = 1$, entonces $t_{jk} \ge t_{ik} + s_i + \tau_{ij}$. Si $y_{ik} = 1$, se exige $a_i \le t_{ik} \le b_i$.

• Se prohíben asignaciones incompatibles con refrigeración y se limita la duración de cada jornada.

Se puede minimizar costo total, usando vehículos adicionales solo cuando sean necesarios. Las entregas críticas se imponen como restricciones; no se intercambian por ahorro mediante una penalización arbitraria.

Ejemplo verificable

Hay dos furgones de capacidad 5 y tres clínicas con demandas A=3, B=2 y C=4. Un vehículo puede atender A y B; el otro, C. Los viajes depósito–A, A–B y B–depósito duran 10, 5 y 12 minutos: la primera ruta requiere 27 minutos de viaje, más descargas. Si A y B deben recibir a la misma hora y los servicios son largos, esa agrupación podría dejar de ser factible pese a respetar la capacidad.

Mezcla, dificultad y uso práctico

El costo de seleccionar una clínica depende de con cuáles se agrupa y en qué orden. Por eso conviene integrar asignación y ruteo. La familia general contiene TSP como caso particular y es NP-hard. No se afirma que toda variante especial de reparto sea difícil.

La salida debe mostrar rutas, cargas, horas y entregas pendientes autorizadas. En instancias grandes pueden combinarse heurísticas de ruteo con MIP para mejorar subproblemas. Deben validarse tiempos de servicio, tráfico, márgenes térmicos y contingencias; un buen recorrido nominal puede ser frágil ante retrasos.

2. Turnos de personal hospitalario

Situación y decisiones

Un hospital debe cubrir turnos de una semana con diferentes competencias. Cada trabajador tiene disponibilidad, máximo de horas y reglas de descanso. La decisión es asignar personas o patrones semanales sin dejar puestos descubiertos ni violar condiciones laborales.

Datos y estructura fundacional

Sean E los trabajadores y R los requisitos de cobertura: cada requisito identifica día, turno y competencia. Se conoce una demanda d_r para cada requisito. Para cada trabajador e se genera un conjunto P_e de patrones legales; cada patrón ya respeta sus descansos y disponibilidad.

Un patrón cubre varios requisitos. Elegir patrones recuerda a Set Cover, pero cuando hacen falta varias personas por puesto se trata de cobertura con multiplicidades. La condición de elegir como máximo un patrón por persona añade una estructura de asignación. Las incompatibilidades entre asignaciones pueden expresarse como conflictos, pero no todo problema de turnos se reduce a coloreo simple.

Variables, restricciones y objetivo

• $z_{ep} = 1$ si el trabajador e usa el patrón p.

• $a_{epr} = 1$ si ese patrón aporta una unidad al requisito r.

• Para cada trabajador: $\Sigma_{p \in P_e} z_{ep} \leq 1$. Puede añadirse un patrón de descanso y exigir igualdad.

• Para cada requisito: $\Sigma_e \Sigma_p a_{epr} z_{ep} \geq d_r$.

• Las competencias se incorporan al generar patrones o al fijar a_{epr}. Una persona no debe contarse simultáneamente en dos puestos incompatibles.

• Se agregan límites globales y reglas entre patrones si no quedaron incorporadas en su generación.

El objetivo puede minimizar costo y, entre soluciones de costo aceptable, mejorar equidad y preferencias. Una formulación por prioridades evita que muchas preferencias satisfechas compensen falta de personal obligatorio.

Ejemplo verificable

Un día requiere dos personas en la mañana y una en la noche. Ana solo puede trabajar de mañana; Bruno puede cubrir cualquiera de los dos turnos, pero no ambos; Carla solo puede cubrir la noche. Los patrones seleccionados Ana-mañana, Bruno-mañana y Carla-noche cubren la demanda. Si Carla falta, el conjunto de personal disponible no permite cubrir ambos requisitos bajo esas reglas: el sistema debe informar inviabilidad, no inventar una asignación.

Mezcla, dificultad y uso práctico

La variante general de selección de patrones contiene Set Cover: con requisitos unitarios, patrones arbitrarios y un candidato independiente por patrón, minimizar patrones equivale a cubrir un universo. Esta observación justifica la dificultad de esa formulación general; no demuestra que cualquier calendario con reglas restringidas sea NP-hard. Algunas asignaciones simples se resuelven por flujo en redes.

La salida incluye el calendario y un informe de cobertura. MIP y programación por restricciones permiten expresar muchas reglas. La equidad debe medirse en varios períodos, porque una semana equilibrada puede ocultar acumulación de noches o fines de semana. También hay que conservar capacidad de respuesta ante ausencias.

3. Distribución de trabajos en una plataforma de cómputo

Situación y decisiones

Una plataforma recibe trabajos de procesamiento con duración estimada, consumo de memoria y requisitos de GPU. Debe decidir qué trabajos ejecutar, dónde colocarlos y cuándo iniciarlos. Puede interesar terminar todos pronto, reducir servidores activos o priorizar trabajos valiosos.

Datos y estructura fundacional

Sean J los trabajos y M los servidores. Se conocen tiempos p_{jm}, memoria r_j, compatibilidad y fechas límite. Si todos los trabajos son obligatorios, los servidores son idénticos y solo se reparte carga, se obtiene minimización del makespan. Si son servicios persistentes que deben caber simultáneamente en servidores de capacidad limitada, aparece empaquetamiento, Bin Packing. Si algunos trabajos pueden rechazarse por un presupuesto de horas, surge mochila.

Estas son variantes relacionadas, pero no intercambiables: la memoria se ocupa durante intervalos, mientras el tiempo de procesamiento se acumula en cada máquina.

Variables, restricciones y objetivo

En el modelo básico de makespan, $x_{jm} = 1$ indica la asignación y C es la duración total:

- Para todo trabajo j: $\Sigma_m\, x_{jm} = 1$.
- Para todo servidor m: $\Sigma_j\, p_{jm} x_{jm} \le C$.
- Se fija $x_{jm} = 0$ cuando el servidor no es compatible.
- Se minimiza C.

Este modelo supone ejecución secuencial por servidor, sin fechas de liberación ni precedencias. Si hay concurrencia, se introducen tiempos de inicio y restricciones acumulativas: en cada instante, la suma de memoria de los trabajos activos no puede superar la capacidad. Para servicios persistentes, una condición distinta sería $\Sigma_j\, r_j x_{jm} \le R_m$.

Ejemplo verificable

Cuatro trabajos duran 8, 7, 6 y 5 minutos. En dos servidores idénticos, agrupar 8+5 y 7+6 produce cargas 13 y 13. Es óptimo porque el total es 26 y el máximo no puede ser inferior a 26/2. Si el trabajo de duración 8 exige GPU y solo un servidor tiene GPU, esa asignación debe comprobarse. Si además dos trabajos compiten por memoria, hay que verificar su solapamiento temporal.

Mezcla, dificultad y uso práctico

La plataforma general contiene scheduling de máquinas idénticas como caso particular, por lo que es NP-hard. Incluso con dos máquinas, decidir si la carga puede equilibrarse exactamente contiene Partición. Pero asignar trabajos divisibles sin otros límites puede admitir soluciones mucho más simples.

La salida es un plan de asignación y ejecución, con tiempos y recursos. Una heurística útil ordena trabajos grandes primero y los asigna a la máquina compatible menos cargada; después pueden mejorarse intercambios. En producción se necesitan estimaciones de duración, colas, reinicios y costos de mover datos. Optimizar solo CPU puede empeorar el tiempo real por congestión de almacenamiento o red.

4. Selección de pruebas de software bajo presupuesto

Situación y decisiones

Antes de publicar una versión, un equipo dispone de diez minutos para ejecutar pruebas. Cada prueba tarda cierto tiempo y detecta o ejercita determinadas condiciones. Debe seleccionar pruebas, reservar las obligatorias y distribuirlas entre ejecutores.

Datos y estructura fundacional

Sean T las pruebas y U los requisitos o escenarios. $a_{ij} = 1$ indica que la prueba i cubre el requisito j. Se conocen duraciones d_i, pesos de riesgo w_j y un presupuesto B. Cubrir todos los requisitos minimizando pruebas corresponde a Set Cover. Maximizar cobertura con un límite de cantidad corresponde a Maximum Coverage; si las duraciones son diferentes, se obtiene su variante presupuestada, relacionada con mochila.

La cobertura debe definirse: líneas ejecutadas, requisitos verificados y fallas detectables son conceptos diferentes. Cubrir código no prueba que el programa sea correcto.

Variables, restricciones y objetivo

- $x_i = 1$ si se ejecuta la prueba i; $y_j = 1$ si se cuenta como cubierto el requisito j.
- Presupuesto secuencial: $\Sigma_i d_i x_i \leq B$.
- Cobertura: $y_j \leq \Sigma_i a_{ij} x_i$.
- Las pruebas obligatorias cumplen $x_i = 1$.
- Se maximiza $\Sigma_j w_j y_j$. Con pesos no negativos, el objetivo activa toda cobertura útil; si hay pesos cero, puede calcularse la cobertura exacta al reconstruir la salida.

Si hay varios ejecutores y el límite es tiempo de reloj, sumar duraciones ya no representa ese límite. Debe añadirse una asignación temporal, dependencias y restricciones de uso exclusivo de entornos. Aparece una mezcla con makespan.

Ejemplo verificable

A dura 6 minutos y cubre {login,pagos}; B dura 4 y cubre {pagos,reportes}; C dura 4 y cubre {login,búsqueda}. Con presupuesto 8 y pesos iguales, B+C cubren cuatro requisitos. A sola cubre dos y no puede combinarse con otra. Elegir primero la prueba con mayor cobertura absoluta no distingue bien estas alternativas.

Mezcla, dificultad y uso práctico

Con duraciones unitarias y pesos iguales, el modelo contiene Maximum Coverage; por eso su versión general es NP-hard. Las interacciones de recursos hacen que una selección con buena cobertura pueda no terminar a tiempo.

La salida debe incluir pruebas, cobertura esperada y cronograma. Puede usarse selección voraz por aporte marginal/costo seguida de mejoras, o MIP cuando la instancia lo permita. Una razón voraz por sí sola no hereda automáticamente todas las garantías del algoritmo de cardinalidad. Deben incorporarse pruebas inestables, preparación de entornos y la posibilidad de detener el proceso al detectar una falla crítica.

5. Ubicación de servicios de emergencia

Situación y decisiones

Una ciudad quiere abrir hasta tres bases de respuesta para atender barrios dentro de un tiempo máximo. Cada ubicación tiene costo y capacidad. La decisión incluye dónde abrir y cómo asignar demanda, considerando que una base cercana puede estar ocupada.

Datos y estructura fundacional

Sean I las ubicaciones y J las zonas. Se conocen poblaciones o demanda d_j, costos f_i y tiempos τ_{ij}. $a_{ij} = 1$ si τ_{ij} no supera el umbral de servicio. Una ubicación cubre un conjunto de zonas. Elegir p ubicaciones para maximizar población cubierta es Maximum Coverage. Si hay capacidades y asignación de demanda, se mezcla con localización de instalaciones y asignación.

Variables, restricciones y objetivo

En el modelo inicial, x_i indica apertura y y_j cobertura:

- $\Sigma_i x_i \le p$ y, si corresponde, $\Sigma_i f_i x_i \le B$.
- $y_j \le \Sigma_i a_{ij} x_i$.
- Maximizar $\Sigma_j d_j y_j$.

Para modelar capacidad, se introduce z_{ij}, demanda de j atendida por i. Se exige $\Sigma_j z_{ij} \le Q_i x_i$ y $z_{ij} = 0$ cuando $a_{ij} = 0$. Si se exige atender toda la demanda de una zona declarada cubierta: $\Sigma_i z_{ij} = d_j y_j$. Este modelo permite repartir demanda entre bases; si no es admisible, deben usarse asignaciones binarias.

Ejemplo verificable

Las zonas A, B, C y D tienen 100, 80, 60 y 40 unidades de demanda. Una base Norte cubre {A,B}; Sur cubre {B,C,D}; Centro cubre {A,C}. Con una base y sin capacidad, Sur cubre 180 unidades y Norte también 180. Con dos bases, Norte+Sur cubren 280. Pero si cada base solo admite 100 unidades, esa cobertura completa es imposible: la demanda total 280 supera la capacidad conjunta 200.

Mezcla, dificultad y uso práctico

Ignorando capacidades y costos distintos se recupera Maximum Coverage, lo que justifica NP-dificultad para la familia general. Añadir restricciones de equidad puede cambiar la solución: maximizar población total podría excluir siempre a una zona pequeña y aislada. Puede exigirse un nivel mínimo por distrito o priorizar primero el peor tiempo de atención.

La salida incluye bases, asignaciones y demanda no atendida. MIP, búsqueda local y análisis de escenarios son opciones habituales. Un radio nominal de cobertura no es una garantía de respuesta: hay tráfico, disponibilidad de vehículos y llamadas simultáneas. Para cubrir fallas de una base puede exigirse respaldo desde dos ubicaciones y evaluar escenarios de indisponibilidad.

6. Configuración de productos y paquetes de software

Situación y decisiones

Un cliente configura un producto con módulos opcionales. Algunas opciones requieren otras, ciertas parejas son incompatibles y hay límites de costo, memoria o energía. La empresa busca una configuración válida que maximice utilidad o minimice costo.

Datos y estructura fundacional

Cada módulo i tiene costo c_i, consumo r_i y utilidad v_i. Se conocen reglas de dependencias y alternativas. La parte de compatibilidad es SAT; la selección bajo recursos se parece a mochila. Ambas se integran mediante SAT con restricciones adicionales, optimización booleana o MIP.

No basta filtrar incompatibilidades por pares: «si se instala A, debe incluirse B o C» es una condición lógica de más de dos elementos.

Variables, restricciones y objetivo

$x_i = 1$ indica que se selecciona el módulo i:

- Dependencia $A \Rightarrow B$: $x_A \leq x_B$.
- Incompatibilidad A y C: $x_A + x_C \leq 1$.
- Al menos uno de B y C: $x_B + x_C \geq 1$.
- Exactamente uno: $x_B + x_C = 1$.
- Dependencia $A \Rightarrow$ (B o C): $x_A \leq x_B + x_C$.
- Presupuesto: $\Sigma_i c_i x_i \leq B$; memoria: $\Sigma_i r_i x_i \leq R$.

Se puede maximizar $\Sigma_i v_i x_i$. Los costos y utilidades aditivos son una simplificación: un paquete con descuento, o dos módulos cuyo beneficio aparece solo al combinarse, requiere variables e interacciones adicionales.

Ejemplo verificable

A es analítica avanzada, cuesta 5 y vale 9; B es el motor requerido, cuesta 3 y vale 2; C es un módulo ligero, cuesta 4 y vale 6. A requiere B y es incompatible con C. Con presupuesto 7, A no puede instalarse, porque A+B cuesta 8. B+C es válido, cuesta 7 y ofrece valor 8, superior a B o C por separado. Ignorar dependencias habría recomendado A por su utilidad individual.

Mezcla, dificultad y uso práctico

Con reglas booleanas generales, la factibilidad contiene SAT. Sin reglas y con una sola capacidad se recupera mochila 0/1. Son dos justificaciones separadas de la dificultad del modelo general. No toda familia de dependencias es igual de difícil: 2-SAT, por ejemplo, es polinómico.

La salida debe explicar tanto lo elegido como las opciones descartadas por reglas. SAT o MIP pueden producir configuraciones; los mecanismos de diagnóstico pueden ayudar a identificar subconjuntos incompatibles de requisitos. Antes de usar recomendaciones conviene comprobar licencias, versiones, consumos reales y módulos obligatorios que no dependen del cliente.

7. Horarios universitarios y selección de oferta académica

Situación y decisiones

Una universidad debe decidir qué cursos ofrecer y asignarles bloque horario y sala. Hay estudiantes compartidos, docentes con disponibilidad, capacidades de salas y competencias que el programa debe cubrir.

Datos y estructura fundacional

Sean C los cursos, T los bloques y R las salas. Se conocen matrícula estimada, capacidad de cada sala, disponibilidad y pares de cursos en conflicto. También se conoce qué competencias cubre cada curso. El núcleo de separación horaria es coloreo: cada curso es un vértice y cada bloque un color. Cubrir competencias con la oferta seleccionada introduce Set Cover; límites de presupuesto y docentes agregan selección y asignación.

Variables, restricciones y objetivo

- $o_c = 1$ si se ofrece el curso c.
- $x_{ctr} = 1$ si se ofrece en el bloque t y la sala r.
- Para cada curso: $\Sigma_t \Sigma_r x_{ctr} = o_c$.
- Para cada combinación sala-bloque: $\Sigma_c x_{ctr} \leq 1$.
- Para cada par conflictivo c,d y bloque t: $\Sigma_r x_{ctr} + \Sigma_r x_{dtr} \leq 1$.
- Se anulan asignaciones por falta de capacidad o disponibilidad.
- Para cada competencia h: $\Sigma_c a_{ch} o_c \geq 1$; pueden requerirse varias alternativas de curso.
- Se limita el presupuesto de oferta y la carga docente.

Este modelo supone una sesión por curso; cursos con varias sesiones requieren patrones semanales y condiciones de separación entre días. El objetivo puede priorizar cursos obligatorios y luego minimizar conflictos residuales y horarios poco convenientes.

Ejemplo verificable

Los cursos A, B y C comparten estudiantes entre cada pareja: forman un triángulo. Se necesitan al menos tres bloques, aunque haya diez salas. Si A cubre programación y B y C son alternativas para cubrir estadística, permitir elegir solo B o C podría reducir la necesidad de bloques; pero debe comprobarse que el plan siga satisfaciendo requisitos de titulación y demanda.

Mezcla, dificultad y uso práctico

Si todos los cursos son obligatorios, hay salas suficientes y la única condición es separar pares conflictivos, decidir si bastan k bloques recupera coloreo de grafos. La familia general es NP-hard. Resolver primero la selección de oferta puede producir cursos baratos imposibles de programar; resolver primero horarios puede consumir recursos necesarios para competencias obligatorias.

La salida es una oferta con calendario, salas y cobertura curricular. Programación por restricciones y MIP son alternativas. La demanda es incierta: puede requerirse abrir secciones adicionales o reservar capacidad. Para validar el modelo debe revisarse que una arista represente una incompatibilidad obligatoria y no una preferencia de algunos estudiantes.

8. Producción con cambios de herramienta y limpieza

Situación y decisiones

Una fábrica recibe pedidos que deben procesarse en máquinas. Cambiar de producto exige preparación, limpieza o cambio de herramienta. Se decide qué producir, en qué máquina y en qué orden. Los tiempos de cambio dependen de la transición y pueden ser asimétricos.

Datos y estructura fundacional

Sean J los trabajos y M las máquinas. Cada trabajo tiene duración p_{jm}, fecha de entrega d_j y máquinas compatibles. s_{ijm} es la preparación si j sigue inmediatamente a i. Ordenar todos los trabajos en una máquina minimizando cambios se relaciona con TSP o Traveling Salesman Path; distribuirlos entre varias añade scheduling y asignación. Varias operaciones por pedido introducen precedencias, como en Job Shop.

Variables, restricciones y objetivo

- x_{jm} indica la máquina asignada; S_j y C_j son inicio y finalización.
- Cada trabajo obligatorio se asigna una vez a una máquina compatible.
- En su máquina: $C_j = S_j + p_{jm}$.
- Cada máquina procesa como máximo un trabajo a la vez.
- Si j es sucesor inmediato de i, entonces $S_j \geq C_i + s_{ijm}$.
- La relación de sucesión debe formar una secuencia conectada por máquina, sin ciclos aislados.
- Si una operación j depende de i: $S_j \geq C_i$.
- La tardanza es $T_j \geq C_j - d_j$ y $T_j \geq 0$.

Puede minimizarse una combinación explícita de makespan, costo de preparación y tardanza. Si las entregas tienen prioridad contractual, conviene optimizarlas antes que pequeños ahorros de preparación. En una implementación hay que definir si la limpieza ocupa también otro recurso, como un equipo compartido.

Ejemplo verificable

Tres trabajos A, B y C duran 2 horas cada uno. Los cambios A→B y B→C requieren 1 hora; los demás cambios entre trabajos distintos, 5. Sin costo inicial, A→B→C tarda 2+1+2+1+2 = 8 horas. A→C→B tarda 16. Si B debe terminar antes de la hora 3, el primer orden no cumple: B termina en la hora 5. El mejor orden por preparación no siempre respeta entregas.

Mezcla, dificultad y uso práctico

Con una máquina, todos los trabajos obligatorios y costos arbitrarios entre sucesores, minimizar preparación reproduce una versión del camino del viajante; si se exige volver a un estado inicial, reproduce un circuito TSP. La formulación general es NP-hard. Con preparaciones uniformes y otras restricciones ausentes, algunos objetivos admiten reglas mucho más simples.

La salida es una secuencia por máquina con tiempos, cambios y tardanzas. Se pueden combinar búsqueda local sobre órdenes y programación por restricciones. Deben modelarse mantenimiento, materiales disponibles y duración variable de procesos; secuencias muy ajustadas pueden ser difíciles de ejecutar ante averías.

9. Campañas de información y selección de promotores

Situación y decisiones

Una institución quiere difundir información mediante promotores. Cada uno tiene costo, una audiencia directa y posibles conexiones de propagación. Se decide a quién contactar y qué públicos priorizar con presupuesto limitado.

Datos y estructura fundacional

Sean V las personas o grupos y A sus relaciones dirigidas. Se conoce el costo c_i de contactar cada semilla y un presupuesto B. Si solo interesa la audiencia directa conocida, cada promotor cubre un conjunto: Maximum Coverage con presupuesto. Si las personas activadas retransmiten el mensaje, aparece Influence Maximization. No deben sumarse audiencias individuales porque pueden superponerse.

Variables, restricciones y objetivo

Para cobertura directa se usan x_i, selección de promotores, e y_j, alcance de cada persona:

- $\Sigma_i\, c_i x_i \le B$.
- $y_j \le \Sigma_i\, a_{ij} x_i$, donde a_{ij} indica alcance directo.
- Maximizar $\Sigma_j\, w_j y_j$, con pesos por prioridad definida.

Para cascadas, el objetivo pasa a ser maximizar σ(S), el número esperado de personas activadas desde $S = \{i : x_i = 1\}$. Es necesario especificar probabilidades, reglas de activación y horizonte. Una aproximación consiste en generar escenarios de propagación, calcular el alcance por escenario y maximizar su promedio. Eso es una estimación del modelo, no una predicción cierta.

Ejemplo verificable

A llega directamente a {1,2,3,4}; B a {3,4,5,6}; C a {7,8,9}. Todos cuestan lo mismo y pueden elegirse dos. A+B cubren seis personas; A+C y B+C cubren siete. Elegir los dos promotores con más audiencia individual falla. Si la persona 2 retransmite a un grupo amplio, A podría aportar más de lo que muestra la cobertura directa; hace falta el modelo de cascada para evaluarlo.

Mezcla, dificultad y uso práctico

El caso de audiencias directas y costos iguales contiene Maximum Coverage. La difusión general es NP-hard y, en modelos probabilísticos comunes, evaluar exactamente σ(S) también puede ser difícil. Las garantías del método voraz requieren condiciones: en modelos clásicos con influencia monótona y submodular y presupuesto de cardinalidad, existe una garantía $1-1/e$ con evaluación exacta. Costos heterogéneos y estimación de influencia requieren análisis adicional.

La salida incluye promotores, alcance estimado y variabilidad. Es razonable contrastar estrategias mediante simulación y pruebas piloto. Las probabilidades pueden cambiar y los grupos pequeños quedar desatendidos por maximizar audiencia total; puede ser necesario exigir cobertura mínima por comunidad. Los datos utilizados deben permitir una representación legítima y apropiada del público.

10. Selección de áreas para conservación ambiental

Situación y decisiones

Una organización dispone de presupuesto para proteger parcelas. Cada parcela contiene hábitats y especies, tiene costo de adquisición y puede conectar otras áreas. Se decide qué parcelas proteger para representar biodiversidad y mantener corredores funcionales.

Datos y estructura fundacional

Sean I las parcelas y S las especies o tipos de hábitat. a_is = 1 indica presencia relevante de s en i; c_i es el costo y w_s la prioridad. Las adyacencias forman un grafo espacial. Maximizar especies representadas bajo presupuesto corresponde a cobertura máxima presupuestada. Si se exige representar todas minimizando costo, aparece Set Cover ponderado. Exigir conexión añade una estructura de selección conectada, relacionada con problemas de redes como Steiner.

La presencia no equivale a viabilidad: proteger una parcela con un individuo no garantiza conservar una población. El modelo básico mide representación, no supervivencia.

Variables, restricciones y objetivo

- $x_i = 1$ si se protege la parcela i; y_s = 1 si se representa la especie s.
- $\Sigma_i c_i x_i \leq B$.
- $y_s \leq \Sigma_i a_{is} x_i$.
- Maximizar Σ_s w_s y_s.

Para proteger una especie en al menos r_s parcelas: $\Sigma_i a_{is} x_i \geq r_s y_s$. Se pueden exigir mínimos de superficie o calidad por hábitat. Para conectividad se fija un área raíz protegida y se envía flujo desde ella a cada parcela seleccionada: el flujo solo atraviesa parcelas elegidas. Deben definirse capacidades de flujo adecuadas y conservación; exigir al menos un vecino elegido no basta, porque permite varios grupos desconectados.

Ejemplo verificable

P1 cuesta 3 y representa {A,B}; P2 cuesta 3 y representa {C,D}. No son adyacentes. Una parcela corredor P3 cuesta 2 y las conecta, pero no añade especies a esta lista. Con presupuesto 6, el modelo de cobertura elige P1+P2 y representa cuatro especies. Si se exige conexión, esa solución necesita P3 y cuesta 8: deja de ser factible. El corredor tiene valor estructural aunque su aporte de cobertura sea cero.

Mezcla, dificultad y uso práctico

Sin conectividad y con costos iguales se recupera Maximum Coverage, por lo que la familia general es NP-hard. Optimizar cobertura primero y añadir corredores después puede superar el presupuesto. Es mejor integrar las decisiones o reservar explícitamente recursos de conexión.

La salida es un portafolio de parcelas con especies representadas, corredores y costo. MIP, búsqueda local y escenarios ecológicos ayudan a comparar alternativas. Deben verificarse calidad de hábitat, movimiento real de especies, incendios, acuerdos de manejo y cambio climático. La adyacencia geométrica es una primera aproximación: un camino o una barrera puede impedir conectividad funcional.

Cómo combinar problemas sin construir un modelo inmanejable

Integración completa. Un modelo contiene selección, asignación y secuenciación. Permite coordinar decisiones, pero puede ser grande. Es útil cuando las interacciones cambian mucho la solución y la instancia tiene tamaño abordable.

Descomposición con retroalimentación. Un modelo principal elige candidatos; otro comprueba rutas o calendarios. Si la decisión no puede ejecutarse, devuelve información para corregirla. Esta interacción puede ser mucho mejor que seleccionar una sola vez y aceptar cualquier resultado posterior.

Horizonte móvil. Se planifica un período, se ejecuta una parte y se recalcula con información nueva. Es apropiado para tráfico, nuevas tareas y ausencias. Debe considerarse el costo de cambiar decisiones ya comunicadas.

Comparación con una referencia. Antes de aumentar complejidad, comparar con una regla sencilla: ruta por cercanía, trabajos grandes primero o cobertura marginal. Medir factibilidad, objetivo, tiempo de cálculo y estabilidad. Una mejora numérica pequeña puede no justificar una operación mucho más frágil.

Lista de revisión para cualquier aplicación

1. ¿Cuáles son las decisiones y cuáles los datos?
2. ¿Qué condiciones son obligatorias y qué preferencias pueden negociarse?
3. ¿Las cantidades se suman o existen superposiciones e interacciones?
4. ¿El modelo cuenta vértices, aristas, personas únicas o unidades de demanda?
5. ¿Qué caso particular permite reconocer un problema fundacional?
6. ¿Cómo se reconstruye una acción real desde las variables?
7. ¿Qué significa una respuesta inviable y cómo se informa?
8. ¿Se necesita optimalidad, una cota de calidad o una solución rápida?
9. ¿Qué incertidumbre podría invalidar el plan?
10. ¿Con qué datos y ejemplos pequeños se verificará el modelo?

Actividad de cierre

Elegir una aplicación del documento y elaborar una instancia con cinco a diez decisiones. Entregar los datos, las variables, las restricciones y el objetivo. Resolver a mano una alternativa factible, explicar por qué una alternativa tentadora falla y añadir una condición realista que obligue a modificar el modelo. Finalmente, proponer un método de resolución y justificar qué calidad de resultado puede prometerse.

Referencias para profundizar

• Tim Roughgarden. *Algorithms Illuminated, Part 4: Algorithms for NP-Hard Problems*. Problemas fundacionales, reducciones y estrategias algorítmicas.

• Jon Kleinberg y Éva Tardos. *Algorithm Design*. Modelado, NP-completitud y aproximación.

• Michael R. Garey y David S. Johnson. *Computers and Intractability*. Referencia sobre problemas NP-completos y reducciones.

• Laurence A. Wolsey. *Integer Programming*. Formulaciones y métodos de programación entera.

• Paolo Toth y Daniele Vigo, eds. *Vehicle Routing: Problems, Methods, and Applications*, segunda edición. Ruteo y variantes operativas.

• Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Planificación temporal y asignación de recursos.

• David Kempe, Jon Kleinberg y Éva Tardos. “Maximizing the Spread of Influence through a Social Network” (2003). Modelos clásicos de difusión e influencia.

Las referencias orientan el estudio de las familias de problemas; no implican que los ejemplos numéricos de esta guía provengan de esas obras. Para demostrar NP-dificultad de una variante concreta es necesario verificar que el caso particular citado respete exactamente sus restricciones.